

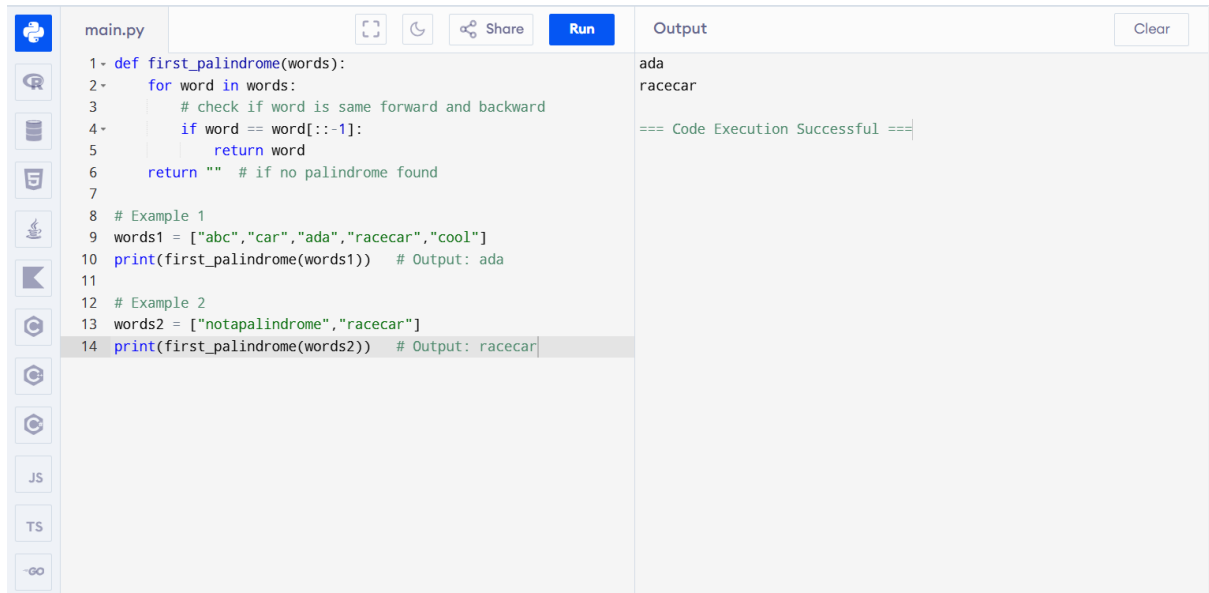
CHAPTER – 1

Name: Ramya S S

Regno: 192421226

Course: CSA0609 – DAA

EXPERIMENT - 1



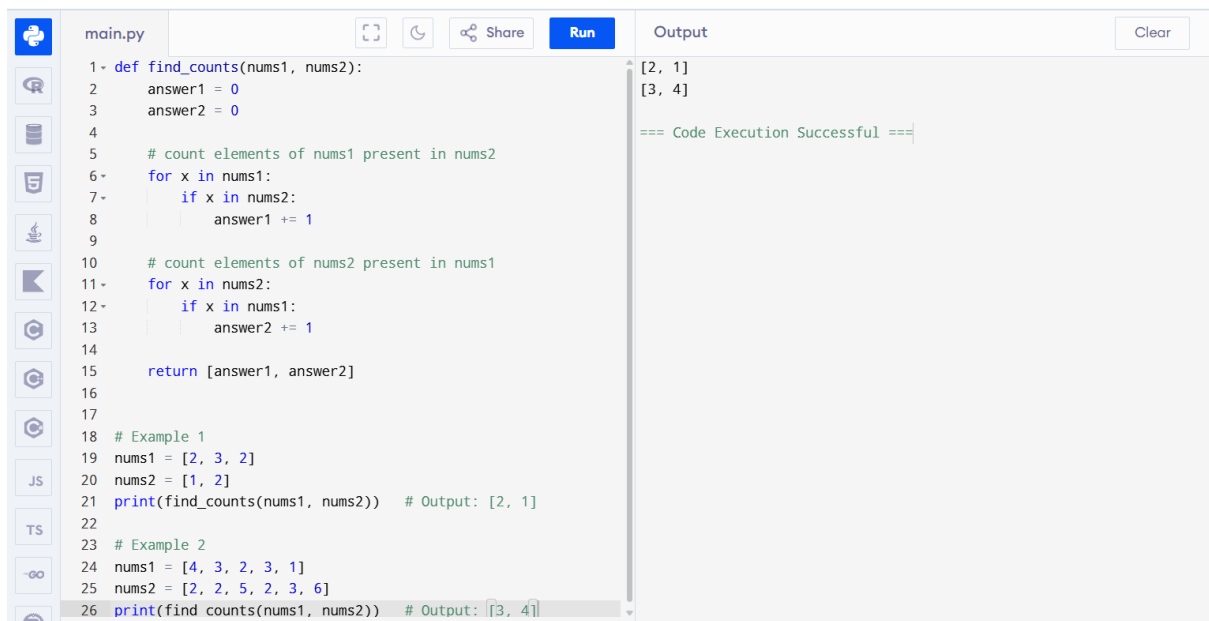
The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'first_palindrome(words)' that iterates through a list of words and returns the first palindrome found. It includes two examples: 'ada' and 'racecar'. The output window shows the results of the function calls.

```
1 def first_palindrome(words):
2     for word in words:
3         # check if word is same forward and backward
4         if word == word[::-1]:
5             return word
6     return "" # if no palindrome found
7
8 # Example 1
9 words1 = ["abc", "car", "ada", "racecar", "cool"]
10 print(first_palindrome(words1)) # Output: ada
11
12 # Example 2
13 words2 = ["notapalindrome", "racecar"]
14 print(first_palindrome(words2)) # Output: racecar
```

Output:

```
ada
racecar
=== Code Execution Successful ===
```

EXPERIMENT – 2



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'find_counts(nums1, nums2)' that counts the number of elements in 'nums1' that are also in 'nums2' and vice versa. It includes two examples: [2, 1] and [3, 4]. The output window shows the results of the function calls.

```
1 def find_counts(nums1, nums2):
2     answer1 = 0
3     answer2 = 0
4
5     # count elements of nums1 present in nums2
6     for x in nums1:
7         if x in nums2:
8             answer1 += 1
9
10    # count elements of nums2 present in nums1
11    for x in nums2:
12        if x in nums1:
13            answer2 += 1
14
15    return [answer1, answer2]
16
17
18 # Example 1
19 nums1 = [2, 3, 2]
20 nums2 = [1, 2]
21 print(find_counts(nums1, nums2)) # Output: [2, 1]
22
23 # Example 2
24 nums1 = [4, 3, 2, 3, 1]
25 nums2 = [2, 2, 5, 2, 3, 6]
26 print(find_counts(nums1, nums2)) # Output: [3, 4]
```

Output:

```
[2, 1]
[3, 4]
=== Code Execution Successful ===
```

EXPERIMENT – 3



main.py



Run

1- def sum_of_squares(nums):
2 n = len(nums)
3 total = 0
4
5 # generate all subarrays
6- for i in range(n):
7 seen = set()
8- for j in range(i, n):
9 seen.add(nums[j]) # add element to set
10 count = len(seen) # distinct count
11 total += count * count # add square
12
13 return total
14
15
16 # Example 1
17 nums1 = [1, 2, 1]
18 print(sum_of_squares(nums1)) # Output: 15
19
20 # Example 2
21 nums2 = [1, 1]
22 print(sum_of_squares(nums2)) # Output: 3

Output

Clear

15
3

=== Code Execution Successful ===

EXPERIMENT – 4



main.py



Run

1- def count_pairs(nums, k):
2 n = len(nums)
3 count = 0
4
5 # check all pairs
6- for i in range(n):
7- for j in range(i + 1, n):
8- if nums[i] == nums[j] and (i * j) % k == 0:
9 count += 1
10
11 return count
12
13
14 # Example 1
15 nums1 = [3,1,2,2,2,1,3]
16 k1 = 2
17 print(count_pairs(nums1, k1)) # Output: 4
18
19 # Example 2
20 nums2 = [1,2,3,4]
21 k2 = 1
22 print(count_pairs(nums2, k2)) # Output: 0

Output

Clear

4
0

=== Code Execution Successful ===

EXPERIMENT – 5

main.py	Output
<pre>1- def find_result(arr): 2 # Using set to remove duplicates 3 unique = set(arr) 4 5 # If all elements are same → return that element 6- if len(unique) == 1: 7 return arr[0] 8 9 # Otherwise return maximum element 10 return max(arr) 11 12 13 # Test Case 1 14 arr1 = [1, 2, 3, 4, 5] 15 print(find_result(arr1)) # Output: 5 16 17 # Test Case 2 18 arr2 = [7, 7, 7, 7, 7] 19 print(find_result(arr2)) # Output: 7 20 21 # Test Case 3 22 arr3 = [-10, 2, 3, -4, 5] 23 print(find_result(arr3)) # Output: 5</pre>	<pre>5 7 5 === Code Execution Successful ===</pre>

EXPERIMENT – 6

main.py	Output
<pre>1- def find_max_after_sort(nums): 2 # check if list is empty 3- if len(nums) == 0: 4 return None # or you can return "List is empty" 5 6 # sort the list (efficient built-in sort → O(n log n)) 7 nums.sort() 8 9 # last element will be maximum 10 return nums[-1] 11 12 13 # Test Case 1: Empty List 14 print(find_max_after_sort([])) # Output: None 15 16 # Test Case 2: Single Element 17 print(find_max_after_sort([5])) # Output: 5 18 19 # Test Case 3: All elements same 20 print(find_max_after_sort([3,3,3,3,3])) # Output: 3</pre>	<pre>None 5 3 === Code Execution Successful ===</pre>

EXPERIMENT – 7

main.py	Output
<pre>1- def unique_elements(nums): 2 unique_list = [] 3 4- for num in nums: 5- if num not in unique_list: 6 unique_list.append(num) 7 8 return unique_list 9 10 11 # Test Case 1: Some duplicates 12 print(unique_elements([3, 7, 3, 5, 2, 5, 9, 2])) 13 # Output: [3, 7, 5, 2, 9] 14 15 # Test Case 2: Negative and positive numbers 16 print(unique_elements([-1, 2, -1, 3, 2, -2])) 17 # Output: [-1, 2, 3, -2] 18 19 # Test Case 3: Large numbers 20 print(unique_elements([1000000, 999999, 1000000])) 21 # Output: [1000000, 999999]</pre>	<pre>[3, 7, 5, 2, 9] [-1, 2, 3, -2] [1000000, 999999] === Code Execution Successful ===</pre>

EXPERIMENT – 8



main.py

Run

Share

Clear

```
1- def bubble_sort(nums):
2   n = len(nums)
3
4   for i in range(n):
5       for j in range(0, n - i - 1):
6           # swap if elements are in wrong order
7           if nums[j] > nums[j + 1]:
8               nums[j], nums[j + 1] = nums[j + 1], nums[j]
9
10          return nums
11
12 # Example usage
13 arr = [5, 3, 8, 4, 2]
14 sorted_arr = bubble_sort(arr)
15 print("Sorted array:", sorted_arr)
```

Output

Clear

Sorted array: [2, 3, 4, 5, 8]

=== Code Execution Successful ===

EXPERIMENT – 9



main.py

Run

Share

Clear

```
1- def binary_search(arr, key):
2   # Step 1: Sort the array
3   arr.sort()
4
5   left = 0
6   right = len(arr) - 1
7
8   while left <= right:
9       mid = (left + right) // 2
10
11      if arr[mid] == key:
12          return mid # position (0-based index)
13      elif arr[mid] < key:
14          left = mid + 1
15      else:
16          right = mid - 1
17
18      return -1
19
20 # Test Case 1
21 arr1 = [3,4,6,-9,10,8,9,30]
22 key1 = 10
23 result1 = binary_search(arr1, key1)
```

Output

Clear

Element 10 is found at position 6

Element 100 is not found

=== Code Execution Successful ===

EXPERIMENT – 10



main.py

Run

Share

Clear

```
1- def merge_sort(nums):
2   if len(nums) <= 1:
3       return nums
4
5   mid = len(nums) // 2
6   left = merge_sort(nums[:mid])
7   right = merge_sort(nums[mid:])
8
9   return merge(left, right)
10
11 def merge(left, right):
12     sorted_list = []
13     i = j = 0
14
15     # Merge two sorted lists
16     while i < len(left) and j < len(right):
17         if left[i] < right[j]:
18             sorted_list.append(left[i])
19             i += 1
20         else:
21             sorted_list.append(right[j])
22             j += 1
23
24     # Add remaining elements
25     sorted_list.extend(left[i:])
26     sorted_list.extend(right[j:])
```

Output

Clear

Sorted array: [1, 2, 3, 5]

=== Code Execution Successful ===

EXPERIMENT – 11

```
main.py  Run  Output  Clear
1- def findPaths(m, n, N, i, j):
2     MOD = 10**9 + 7
3
4     # dp[x][y] = number of ways to reach cell (x, y)
5     dp = [[0 for _ in range(n)] for _ in range(m)]
6     dp[i][j] = 1
7
8     count = 0
9
10    for step in range(N):
11        temp = [[0 for _ in range(n)] for _ in range(m)]
12
13        for x in range(m):
14            for y in range(n):
15                if dp[x][y] > 0:
16                    ways = dp[x][y]
17
18                    # Move in 4 directions
19                    for dx, dy in [(1,0), (-1,0), (0,1), (0,-1)]:
20                        nx, ny = x + dx, y + dy
21
22                        # If out of boundary → count it
23                        if nx < 0 or nx >= m or ny < 0 or ny >= n:
24                            count = (count + ways) % MOD
```

6
12
=== Code Execution Successful ===

EXPERIMENT – 12

```
main.py  Run  Output  Clear
1- def rob(nums):
2     if not nums:
3         return 0
4     if len(nums) == 1:
5         return nums[0]
6
7     # helper function (normal house robber)
8     def rob_line(arr):
9         prev = 0 # dp[i-2]
10        curr = 0 # dp[i-1]
11
12        for num in arr:
13            temp = max(curr, prev + num)
14            prev = curr
15            curr = temp
16
17        return curr
18
19    # Case 1: exclude last house
20    case1 = rob_line(nums[:-1])
21
22    # Case 2: exclude first house
23    case2 = rob_line(nums[1:])
24
25    return max(case1, case2)
```

3
4
=== Code Execution Successful ===

EXPERIMENT – 13

```
main.py  Run  Output  Clear
1- def climb_stairs(n):
2     if n <= 2:
3         return n
4
5     prev1 = 1 # ways to reach step 1
6     prev2 = 2 # ways to reach step 2
7
8     for i in range(3, n + 1):
9         current = prev1 + prev2
10        prev1 = prev2
11        prev2 = current
12
13    return prev2
14
15
16 # Test Cases
17 print(climb_stairs(4)) # Output: 5
18 print(climb_stairs(3)) # Output: 3
```

5
3
=== Code Execution Successful ===

EXPERIMENT – 14



main.py

Run

Clear

```
1- def unique_paths(m, n):
2     dp = [[1] * n for _ in range(m)]
3
4     for i in range(1, m):
5         for j in range(1, n):
6             dp[i][j] = dp[i-1][j] + dp[i][j-1]
7
8     return dp[m-1][n-1]
9
10
11 # Test Cases
12 print(unique_paths(7, 3)) # Output: 28
13 print(unique_paths(3, 2)) # Output: 3
```

Output

28
3

=== Code Execution Successful ===

EXPERIMENT – 15



main.py

Run

Clear

```
1- def largeGroupPositions(s):
2     result = []
3     n = len(s)
4     i = 0
5
6     while i < n:
7         j = i
8
9         # Move j while characters are the same
10        while j < n and s[j] == s[i]:
11            j += 1
12
13        # Check if group length >= 3
14        if j - i >= 3:
15            result.append([i, j - 1])
16
17        i = j # Move to next group
18
19    return result
20
21
22 # Test Cases
23 print(largeGroupPositions("abbxxxxzzy")) # Output: [[3,6]]
24 print(largeGroupPositions("abc")) # Output: []
```

Output

[[3, 6]]
[]

=== Code Execution Successful ===

EXPERIMENT – 16



main.py

Run

Clear

```
1- def gameOfLife(board):
2     m, n = len(board), len(board[0])
3
4     directions = [(-1,-1), (-1,0), (-1,1),
5                  (0,-1),      (0,1),
6                  (1,-1), (1,0), (1,1)]
7
8     for i in range(m):
9         for j in range(n):
10            live_neighbors = 0
11
12            for dx, dy in directions:
13                ni, nj = i + dx, j + dy
14                if 0 <= ni < m and 0 <= nj < n and abs
15                   (board[ni][nj]) == 1:
16                    live_neighbors += 1
17
18            if board[i][j] == 1:
19                if live_neighbors < 2 or live_neighbors > 3:
20                    board[i][j] = -1
21            else:
22                if live_neighbors == 3:
23                    board[i][j] = 2
24
25    for i in range(m):
```

Output

[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]

=== Code Execution Successful ===

EXPERIMENT – 17

main.py		Output
<pre>1 def champagneTower(poured, query_row, query_glass): 2 dp = [[0.0] * (k + 1) for k in range(query_row + 1)] 3 dp[0][0] = poured 4 5 for i in range(query_row): 6 for j in range(len(dp[i])): 7 if dp[i][j] > 1: 8 overflow = (dp[i][j] - 1) / 2 9 dp[i+1][j] += overflow 10 dp[i+1][j+1] += overflow 11 12 return min(1, dp[query_row][query_glass]) 13 14 15 # Test cases 16 print(champagneTower(1, 1, 1)) # 0.0 17 print(champagneTower(2, 1, 1)) # 0.5</pre>	<pre>0.0 0.5 === Code Execution Successful ===</pre>	